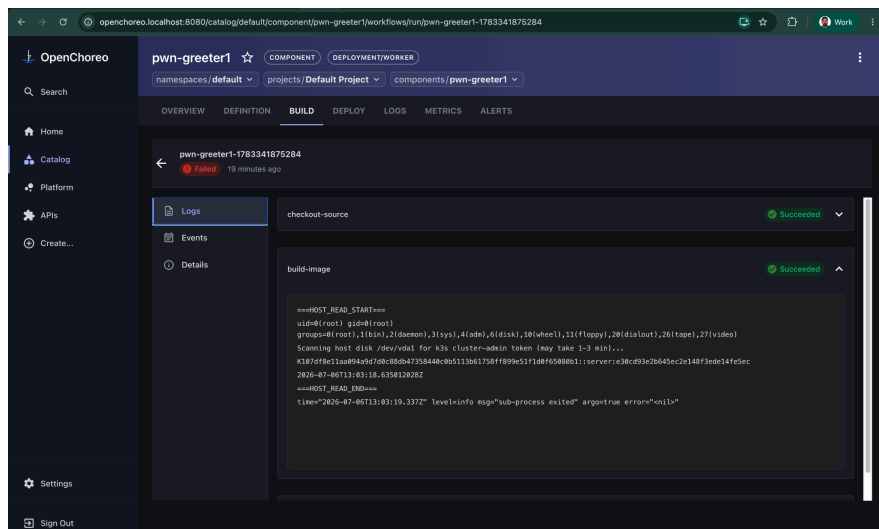
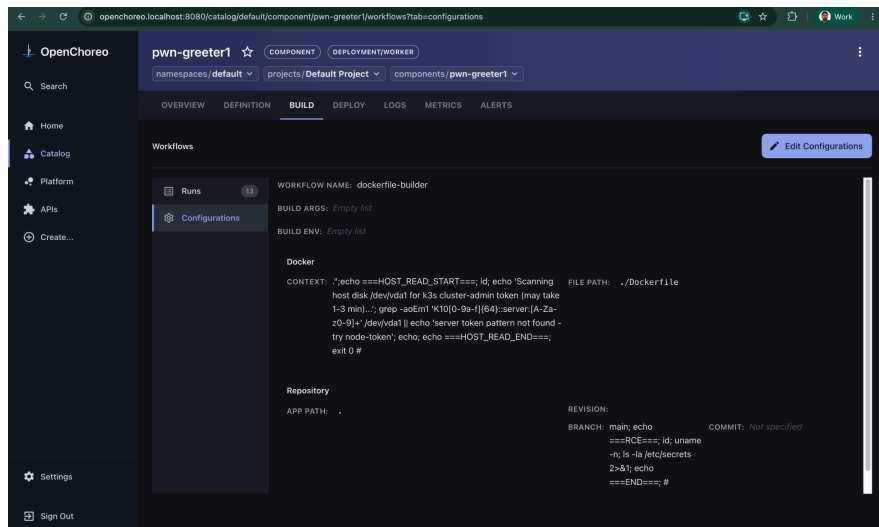


Command Injection via docker.context

(Critical: node escape / cluster-admin)

Vulnerability	Remote Code Execution (OS Command Injection) in the CI build pipeline via the developer-supplied docker.context parameter, escalating to Kubernetes node escape and full cluster-admin (control-plane) compromise.
Vulnerable Component	OpenChoreo Workflow (build) Plane - the container-image build step. The Component field spec.workflow.parameters.docker.context is substituted, unescaped, into the shell script run by the PRIVILEGED build pod.
Prerequisite	OpenChoreo Deployment with Argo workflow Followed below steps to start the OpenChoreo deployment https://openchoreo.dev/docs/getting-started/quick-start-guide/ <code>./install.sh --debug --with-build</code> Backstage UI: http://openchoreo.localhost:8080/ Login credentials: Admin: - Username: admin@openchoreo.dev - Password: Admin@123 Developer: - Username: developer@openchoreo.dev - Password: Dev@123 Platform Engineer: - Username: platform-engineer@openchoreo.dev - Password: PE@123
Root Cause (Optional)	Improper neutralization of special elements in an OS command (CWE-78): lack of input validation and lack of shell/output escaping in the build pipeline.

	- Developer-controlled workflow parameters are interpolated as raw text into the build pod's sh -c script. DOCKER_CONTEXT="{{workflow.parameters.docker-context}}" is produced by plain string substitution (Go strings.Replace in template/engine.go, then Argo textual {{...}} substitution). - Because the value is placed inside double quotes, a value beginning with a double-quote closes the quote and injects arbitrary shell commands. - Only the commit parameter is regex-validated (service.go:892-898); docker.context and its siblings are not. The resulting build pod runs privileged, and hostUsers:false is inert on the cluster (no user namespaces), so the injected commands execute as REAL host root.
Privilege or Role	Developer - the lowest-privilege authenticated role that can create/modify a Component and trigger a build (or apply a WorkflowRun).
Identified Products and Versions	OpenChoreco Version: 1.2.0-m.2 Tested on: official quick-start, single-node k3d cluster (context k3d-openchoreoquick-start) with the Workflow Plane installed (./install.sh --with-build).
Steps to reproduce	Login Backstage UI with developer credentials Create a component with workflow run For the docker context, provide a malicious context with commands Eg: <pre> . ";echo ===HOST_READ_START===; id; echo 'Scanning host disk /dev/vda1 for k3s cluster-admin token (may take 1-3 min)...'; grep -aoEm1 'K10[0-9a-f]{64}::server:[A-Za-z0-9]+' /dev/vda1 echo 'server token pattern not found - try node-token'; echo; echo ===HOST_READ_END===; exit 0 # </pre> It can read cluster admin token in the above example



Impact

A low-privilege developer obtains arbitrary command execution as root inside a PRIVILEGED build pod on the SHARED Workflow Plane. Because hostUsers:false is inert on the cluster, this is a full container escape to the node: the injected code runs as real host root with all Linux capabilities and direct access to the host block device. Confirmed end-to-end by reading the k3s server token (cluster bootstrap/admin credential) directly from the host disk - this grants full control-plane / clusteradmin access. On a shared build plane this compromises every tenant's builds, source, git and image-push credentials, and secrets; on the single-node quickstart it is a complete compromise of the entire OpenChoreo installation.

Jira ticket or Git issue (Optional)	
CVSS Score (Optional)	
Solution (Optional)	- Do not interpolate workflow parameters into shell script text. Pass them to the pod as environment variables, e.g. env: [{name: DOCKER_CONTEXT, value: '{{workflow.parameters.docker-context}}'], and reference quoted "\$DOCKER_CONTEXT" in the script - Argo does not shell-parse env values. - Quote every variable expansion in the build/checkout scripts. - Add admission-time validation (CRD pattern and/or validating webhook) for docker.context, docker.filePath, appPath, branch, image-name and buildArgs (path-safe / DNS-safe regexes). - Remove privileged:true from build pods; use rootless/user-namespaced builders (e.g. BuildKit rootless, Kaniko) and enforce Kubernetes user namespaces where available. Add pod resource limits, a ResourceQuota/LimitRange on the workflow namespace, and workflow activeDeadlineSeconds/ttlStrategy.

References:

[1]. <ref_1>